

CreditGuard MCP

Agentic Credit Analyst Simulator with Rule-Based and AI/MCP Decisioning

Project summary. CreditGuard MCP is a simulated banking credit-card limit review system. It lets a user manually create credit-card transactions, payment records, and risky customer behavior through a Gradio dashboard. The system then evaluates whether a customer's credit limit should be increased, decreased, kept unchanged, or routed to manual review.

Updated note. The project includes both a local Card MCP Server for private simulated banking data and a Web/Economic Research MCP Server using Tavily for external economic and consumer-credit context. The Tavily integration is part of the agentic design, while the project can still run locally without web search if the Tavily key is unavailable.

1. High-Level Architecture

- Gradio Dashboard: user-facing UI for adding transactions, viewing card state, and running analyst workflows.
- SQLite Database: local source of truth for customer, card, transaction, payment, decision, and audit-log records.
- Rule-Based Credit Analyst Simulator: deterministic Python logic used to test the workflow quickly.
- AI-Based Credit Analyst Agent: OpenAI Agents SDK agent that calls MCP tools and saves decisions.
- Card MCP Server: exposes local banking data and action tools to the AI agent.
- Tavily Web/Economic Research MCP Server: exposes external web-search tools for economic and consumer-credit context.

Layer	Responsibility
Gradio dashboard	Adds regular/risky card transactions, adds payments, runs the analyst, and displays dashboard results.
SQLite database	Stores all simulated banking data and analyst outputs.
Rule-based analyst	Applies deterministic rules to create a baseline credit-limit decision.
AI Credit Analyst Agent	Uses MCP tools to gather evidence, optionally research economic context, make a decision, and save results.
Card MCP Server	Reads customer/card/transaction/payment data and writes decisions/audit logs.
Tavily Web Search MCP	Searches external economic and consumer-credit risk information to support the final decision.

2. Two Decision Modes

2.1 Manual Rule-Based Credit Analyst Simulator

The manual simulator does not use an LLM. It calculates a risk score using Python rules based on credit score, annual income, utilization, risky transaction count, and late payment count. It is useful for validating the database, dashboard, and credit-limit decision workflow before introducing AI and MCP.

- Increase: strong credit score, low utilization, no late payments, and low risk indicators.
- Decrease: high utilization, repeated risky behavior, low credit score, or multiple late payments.
- Manual Review: moderate risk or borderline conditions.
- No Change: acceptable profile that does not strongly qualify for an increase.

2.2 AI-Based Credit Analyst Agent

The AI-based agent uses the OpenAI Agents SDK and MCP. It calls tools from the local Card MCP Server and the Tavily Web/Economic Research MCP Server, then saves a decision and writes audit logs. This is closer to the trading simulator pattern because the agent researches, reasons, acts through tools, and updates local state.

- Always gathers local credit profile, recent transactions, and payment history.
- Uses Tavily web search for external consumer-credit, delinquency, household-debt, and interest-rate context.
- Makes one decision: Increase, Decrease, No Change, or Manual Review.
- Calls `save_credit_decision` to persist the decision and update the simulated credit limit.
- Calls `write_audit_log` to make agent activity visible in the dashboard.

3. Main User Flow

1. Start the Gradio dashboard.
2. Add regular credit-card transactions such as Tim Hortons, Walmart, Netflix, or TTC.
3. Add risky transactions such as Unknown Crypto Exchange, Luxury Electronics Store, Foreign Gambling Site, or Cash Advance ATM.
4. Add card payments or late payment records if payment history is enabled.
5. Run either the rule-based Credit Analyst Simulator or the AI-based Credit Analyst Agent.
6. The selected analyst evaluates the account and saves a credit-limit decision.
7. The dashboard refreshes and shows the latest decision, evidence, transactions, payment behavior, and audit logs.

4. Database Tables

Table	Purpose
customers	Stores customer identity and credit profile such as income, employment status, and credit score.
credit_cards	Stores current credit limit, balance, utilization basis, and card status.
card_transactions	Stores simulated card purchases, risky transactions, payments, and reviewed flags.
payment_history	Stores on-time and late payment records used for credit risk evaluation.
credit_limit_decisions	Stores analyst decisions, old/new limits, risk score, reasoning, and timestamp.
audit_logs	Stores manual simulator logs, agent activity, and decision/audit messages.

5. MCP Servers and Tools

5.1 Local Card MCP Server

File: `card_mcp_server.py`. This server connects the AI agent to local simulated banking data stored in SQLite.

MCP tool	What it does	Why the agent uses it
<code>get_credit_profile</code>	Returns customer profile, card limit, current balance, utilization percent, risky transaction count, late payment count, and on-time payment count.	Provides the core creditworthiness evidence for the decision.
<code>get_recent_transactions</code>	Returns recent credit-card transactions with merchant, amount, category, suspicious seed, and reviewed status.	Lets the agent inspect spending behavior and risky transaction patterns.
<code>get_payment_history</code>	Returns recent payment records with status	Lets the agent evaluate repayment behavior,

	and days late.	which is critical for credit-limit decisions.
save_credit_decision	Saves the agent decision, old/new limit, risk score, reasoning summary, and updates the simulated card limit.	This is the action tool; it is equivalent to a trading agent buying/selling in the portfolio simulator.
write_audit_log	Writes an audit/activity message into the audit_logs table.	Makes agent activity visible and auditable on the Gradio dashboard.

5.2 Tavily Web/Economic Research MCP Server

This server gives the Credit Analyst Agent external context. It should be connected as a second MCP server using Tavily or the Tavily remote MCP endpoint. It is used to research public economic and consumer-credit conditions, not private customer data.

Web research area	Example search intent	How it affects the decision
Consumer credit risk	Search current credit-card delinquency trends or consumer credit stress.	Can make the agent more conservative when external risk is elevated.
Interest-rate context	Search current rate environment or central bank policy context.	Can influence whether aggressive credit expansion is appropriate.
Household debt conditions	Search public household debt or debt-service risk updates.	Supports macro-level risk context.
Bank credit tightening	Search whether lenders are tightening credit standards.	May support manual review or smaller increases.

Important design rule: Tavily search is supporting evidence only. The agent should not let web research override severe customer-level risk such as very high utilization, multiple late payments, or low credit score.

6. Dashboard Displays

- CreditGuard Summary: customer name, credit score, current limit, current balance, and utilization.
- Latest Credit Analyst Decision: decision, old limit, new limit, risk score, reasoning, and timestamp.
- Evidence Used by Credit Analyst: structured evidence such as utilization, risky transaction count, and payment counts.
- Recent Card Transactions: regular/risky transaction history.
- Payment History: on-time and late payment records.
- Credit Limit Decisions: decision history.
- Agent/Audit Logs: simulator logs and AI agent activity.
- Utilization Chart: visual chart of current card utilization.

7. Demo Scenarios

Scenario	User action	Expected outcome
Clean customer	Reset demo data and run analyst.	Likely Increase due to strong credit score, low utilization, and no late payments.
Risky spending	Add multiple risky transactions and run analyst.	Manual Review or Decrease depending on utilization and risk score.
Late payments	Add late payment records and run analyst.	Higher risk score; likely Manual Review or Decrease.
Improved balance	Add card payment, reducing utilization, then run analyst.	Risk may improve; decision may move toward No Change or Increase.
Web-enhanced decision	Run AI agent with Tavily MCP enabled.	Decision includes external economic/consumer-credit context in the reasoning summary.

8. Run Commands

Project dependencies are managed with uv.

```
uv sync
```

```
uv run dashboard.py
```

```
uv run credit_analyst_agent.py
```

Required environment variables:

```
OPENAI_API_KEY=your_openai_key_here
```

```
TAVILY_API_KEY=your_tavily_key_here
```

9. Resume Description

Built CreditGuard MCP, an agentic credit-card limit review simulator using Gradio, SQLite, OpenAI Agents SDK, and Model Context Protocol. The system supports both a rule-based Credit Analyst Simulator and an AI-based Credit Analyst Agent that calls local MCP tools to retrieve customer profile data, transactions, and payment history, calls Tavily Web Search MCP for economic/consumer-credit context, saves credit-limit decisions, and writes audit logs. The Gradio dashboard visualizes card utilization, transactions, payment behavior, credit-limit decisions, reasoning summaries, and audit activity.

10. Short Interview Explanation

CreditGuard MCP is a simulated banking credit-limit review system. I first built a rule-based Credit Analyst to validate the workflow, then added an AI-based Credit Analyst Agent using OpenAI Agents SDK and MCP. The agent calls a local Card MCP Server for customer, card, transaction, and payment data, and can also call a Tavily Web Search MCP Server for external economic and consumer-credit context. It decides whether to increase, decrease, keep unchanged, or send the account to manual review, then saves the decision and reasoning to SQLite for display in a Gradio dashboard.